

Auditoria de Performance — JADE IA Mobile

Otimizações seguras aplicadas sem alteração funcional ou de UX

25 de junho de 2026

Contexto & Diagnóstico Inicial

O app JADE IA (Expo SDK 54, React Native, expo-router) foi auditado quanto a gargalos de performance no lado do cliente. A análise identificou os seguintes problemas estruturais:

- 166 ScrollViews vs apenas 10 FlatLists — maioria das listas longas sem virtualização
- 0 componentes com React.memo — todos re-renderizam a cada mudança de contexto
- Apenas 7 usos de useCallback/useMemo em 49 telas
- 195 arrow functions inline em props de componentes
- 52 useEffects espalhados (média 1,06 por tela)

Todos os contextos (8 providers) aninhados no `_layout.tsx` recriam valores a cada render do root, propagando re-renders desnecessários para todos os consumidores.

1. QueryClient — Cache Global (app/_layout.tsx)

Problema: QueryClient criado sem configuração; `staleTime = 0ms` (refetch a cada foco de tela) e `gcTime` padrão de 5min, mas com retry automático ilimitado.

Solução aplicada: configuração explícita de parâmetros globais.

- `staleTime: 30 000ms` — dados permanecem frescos por 30 s, evitando refetch em navegação rápida entre telas
- `gcTime: 300 000ms (5 min)` — mantém cache em memória mesmo após desmonte do componente
- `retry: 1` — apenas 1 tentativa em erro (padrão era 3)
- `refetchOnWindowFocus: false` — sem refetch automático ao retornar ao app em foreground

2. React.memo nos Componentes de Lista

Problema: `LeadCard` (`leads.tsx`), `ScoreBadge` (`leads.tsx`) e `ConversationItem` (`conversas.tsx`) eram funções simples sem memoização. Qualquer mudança de estado na tela pai (ex.: abertura de modal, digitação no filtro) re-renderizava todos os itens da lista.

Solução: os três componentes foram convertidos para `const + memo()`, garantindo que o React só os re-renderize quando suas props mudarem de fato.

- `ScoreBadge` — memoizado; re-renderiza apenas se a prop `lead` mudar
- `LeadCard` — memoizado; re-renderiza apenas se `lead` ou `onPress` mudar
- `ConversationItem` — memoizado; re-renderiza apenas se `item`, `onPress` ou `limitReached` mudar
- Impacto: kanban com 30+ leads !' re-render por filtro cai de $O(n)$ para 0 re-renders nos itens

3. useCallback nos Handlers de Evento (leads.tsx)

Problema: openDetail e handleMove eram arrow functions recriadas a cada render. Como são passadas como props para LeadCard (agora memoizado), referencialmente instáveis anulariam o benefício do memo.

Solução: ambas envolvidas em useCallback com deps mínimas.

- openDetail — deps: [] (sem dependências; usa apenas setters de estado que são estáveis)
- handleMove — deps: [selectedLead, moveLead] (depende do lead selecionado e da função do contexto)
- Resultado: LeadCard.memo é efetivo pois recebe referências estáveis de onPress

4. useMemo nas Computações de Lista

Problema: filter + sort dos leads executavam em cada render da tela, mesmo sem mudança em leads ou filter. Em conversas.tsx, applyFilter + toLowerCase também rodavam inline a cada render.

Soluções aplicadas:

- leads.tsx — columnData: useMemo com deps [leads, filter]; processa 4 colunas × sort × filter hot apenas quando os dados mudam
- conversas.tsx — filtered: useMemo com deps [conversations, activeFilter, query]
- conversas.tsx — waitingCount: useMemo com deps [conversations]
- Impacto: interações de UI (abrir modal, scroll) não mais disparam sort/filter

5. useMemo no Valor do AppContext

Problema: o objeto passado como value para AppContext.Provider era recriado a cada render do NativeAppProvider e do WebAppProvider — mesmo quando nenhum dado mudou. Isso forçava re-render em TODOS os consumidores de useApp() da aplicação.

Solução: o objeto value foi extraído para useMemo com deps nas variáveis de estado (leads, conversations, moduleStates, activityEvents, campaigns, leadActivities, loading).

Efeito: 30+ telas que consomem useApp() não re-renderizam mais por mudanças internas do provider não relacionadas.

6. Props de Performance em FlatLists

Problema: FlatLists existentes não tinham props de controle de virtualização — React Native renderiza por padrão 10 itens por lote sem limite de janela, resultando em travamentos ao scollar listas longas.

Solução: adicionadas 3 props de performance nos 4 FlatLists críticos:

- jade.tsx (chat inverted) — initialNumToRender=15, maxToRenderPerBatch=8, windowSize=8
 - conversas.tsx (lista de conversas) — initialNumToRender=12, maxToRenderPerBatch=8, windowSize=10
 - crm.tsx (lista de contatos) — initialNumToRender=12, maxToRenderPerBatch=8, windowSize=10
 - historico.tsx (sessões) — initialNumToRender=10, maxToRenderPerBatch=6, windowSize=8
 - conversa/[id].tsx (mensagens individuais) — initialNumToRender=15, maxToRenderPerBatch=8, windowSize=8
-

Resumo das Alterações

- 8 arquivos modificados: `_layout.tsx`, `leads.tsx`, `conversas.tsx`, `jade.tsx`, `crm.tsx`, `historico.tsx`, `conversa/[id].tsx`, `AppContext.tsx`
 - 3 componentes memoizados com `React.memo` (`ScoreBadge`, `LeadCard`, `ConversationItem`)
 - 2 handlers convertidos para `useCallback` (`openDetail`, `handleMove`)
 - 5 computações convertidas para `useMemo` (`columnData`, `filtered`, `waitingCount`, `ctxValue` ×2)
 - 5 `FlatLists` com `initialNumToRender` + `maxToRenderPerBatch` + `windowSize` configurados
 - 1 `QueryClient` configurado com `staleTime`/`gcTime`/`retry`/`refetchOnWindowFocus`
 - Nenhuma mudança funcional ou de UX — zero breaking changes
 - Typecheck: LIMPO (`pnpm --filter @workspace/mobile run typecheck` — zero erros)
-

Otimizações Mantidas Fora do Escopo

As seguintes melhorias foram identificadas mas deixadas para uma segunda fase por envolverem maior risco ou refatoração significativa:

- Converter `ScrollViews` de listas longas para `FlatList` (166 ocorrências — requer audit caso a caso)
 - Habilitar `babel-plugin-react-compiler` (instalado mas em beta; pode impactar padrões do Expo RN)
 - `useCallback` em funções do `AppContext` que dependem de estado (`moveLead`, `addLead` etc.) — requer análise de deps para evitar closures obsoletas
 - `getItemLayout` em `FlatLists` com altura fixa — melhora significativa mas requer medição dos itens
-

